

00_Apuntes_Tensorflow_DeepLearning

May 11, 2026

1 Apuntes: Libreria Tensorflow en Python para Deep Learning

TensorFlow es una libreria generada por Google para Python centrada en el entreno de algoritmos y modelos de Deep Learning, que se podrian definir como una herramienta que permite entrenar modelos de prediccion que se retroalimentan.

Antes de comenzar, vamos a indagar que es el Deep Learning y que diferencias tienen con el Machine Learning.

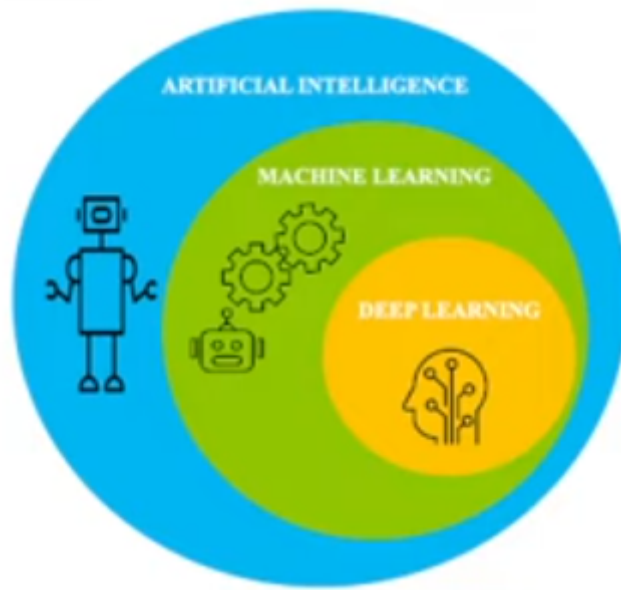
1.1 1.0 ¿Que es el Deep Learning?

Consiste en un conjunto de procesos informaticos que tienen como objetivo imitar el aprendizaje humano y el comportamiento de interacciones que existen en nuestro cerebro con las neuronas, imitando esa interaccion.

El deep learning tiene una potencia mayor para generar ese aprendizaje (*autoaprendizaje*), **dispone de una serie de algoritmos para solventar problemas mas complejos** (clasificacion, regresion de datos,...).

1.1.1 Aspectos a tener en cuenta

- Hay 2 conceptos clave: **el perceptron y la red neuronal artificial (ANN)**
- Hay 2 tipos de algoritmos: **Supervisados** (requieren intervencion humana, supervisando una serie de datos de entrenamiento) y **No supervisados** (centrados en la clsuterizacion, que no requieren de supervision humana para generar sus modelos)



1.1.2 1.1 Conceptos clave: Los Perceptrones y la Red Neuronal Artificial (ANN)

1.1.3 Los Perceptrones

Es una parte del sistema que se dedica a **entender los datos a través de múltiples capas** (*es la cognición perceptiva del sistema*)

El perceptron es la neurona que ayuda a componer la red neuronal

Funciona del siguiente modo:

- **Cada capa intenta entender una parte de los datos de entrada** > Cada capa, se centra en descomponer una parte del problema a resolver, para resolver el problema en su conjunto en problemas más pequeños y sencillos.
- **Cada capa se puede componer de unidades computacionales que aprenden a detectar ocurrencias repetidas de los valores** > Las capas permiten asociarse entre ellas para comprender mejor patrones que ayudan a definir las respuestas que están buscando.
- **Una red de estas capas, imita como funcionan las neuronas y componen las redes neuronales artificiales.**

1.1.4 Red Neuronal Artificial (ANN)

Es la red de perceptrones y algoritmos que imita el cerebro humano para encontrar la relación en una serie de datos.

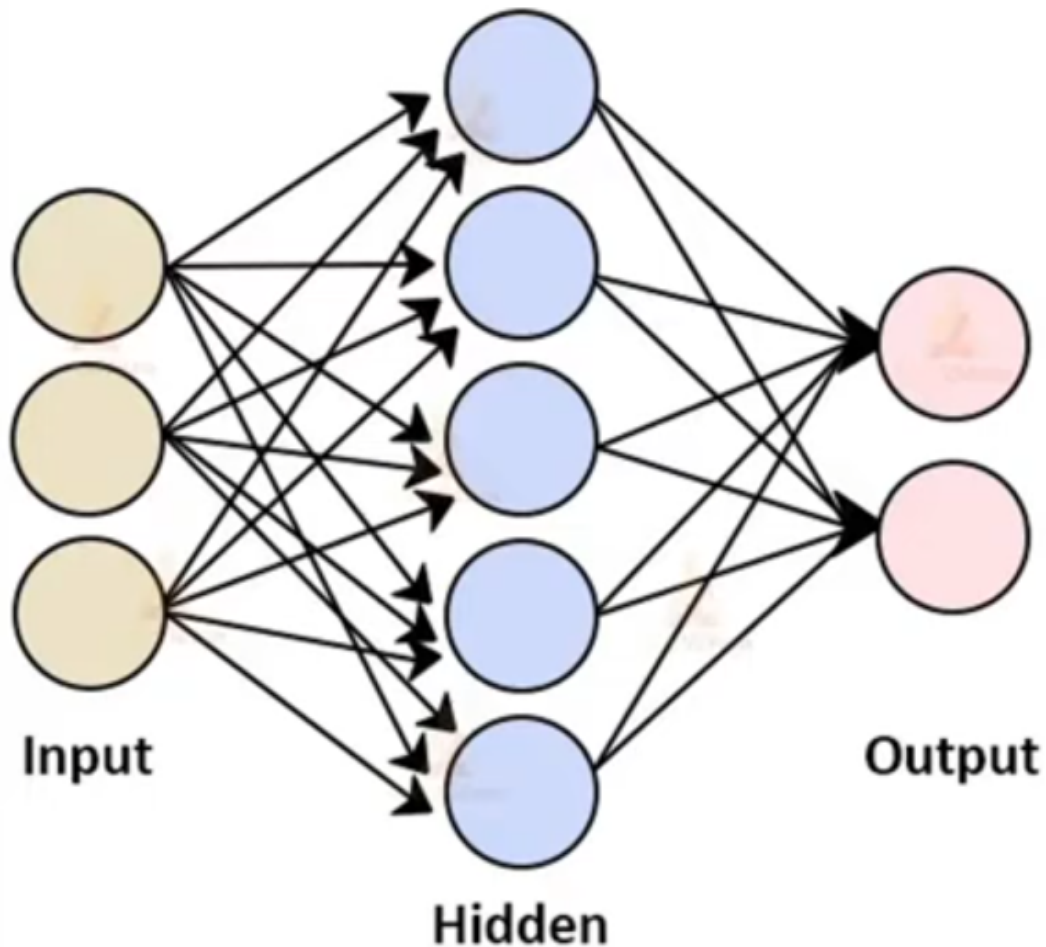
Su aplicación es una variada, desde la clasificación, la regresión, el reconocimiento de imágenes,....

Hay 3 niveles de capas que componen la red:

- **Input Layer:** Son los algoritmos que **toman los datos de entrada y los traspasan a la capa hidden layer**

- **Hidden Layer:** Es donde se realiza la computacion de los datos y los transfieren a la capa de salida > Lugar donde se albergan los perceptrones y ayudan a generar los entender los datos para generar patrones. > > Pueden existir varias capas de este tipo
- **Output Layer:** **Se termina de procesar (*normalizar*) los datos para sacar los resultados

Architecture of Artificial Neural Network



1.2 2.0 Ventajas y desventajas del Deep Learning

Ahora veremos ventajas y desventajas del Deep Learning como concepto, para poder diferenciarlo del Machine Learning.

1.2.1 Ventajas

- Las Redes neuronales pueden aprender modelos no lineales (no hay relacion directa entre los datos) y mas complejos

Tambien se **pueden resolver problemas complejos de manera mas sencilla a nivel de programacion**

- Tras un entrenamiento previo supervisado, **tienen la capacidad de inferir relaciones (transmitir y reprocesar) de este aprendizaje en datos nuevos**

Estos datos tienen que **tener el mismo formato que los utilizamos en el entrenamiento**

- No hay restricciones en los dataset de entrada

Se pueden *normalizar los datos* de los datasets, para que el modelo los maneje

- **Permite tolerar fallos** en partes del procesamiento de datos y **no afecta al resultado final**. Mas aun, los propios modelos que fallan en alguna parte **son capaces de reaprender**.

1.2.2 Desventajas

- **No somos capaces de entender que esta pasando en el procesamiento**

Aunque defines los datos de entrada, los datos de entrenamiento, datos de test, y las capas... muchas veces no somos capaces de entender todo lo que ocurre por detras

- **No hay una regla directa que determine que estructura es mejor para una red neuronal**

Al ser tan diversas y complejas de entender, no es facil discernir cual es la mejor metodologia, **solo mediante experiencia de uso se puede extraer conceptos y desarrollar mejores estructuras**

1.2.3 Deep Learning vs Machine Learning

Tienen ambos en comun que son formas de imitar el aprendizaje humano, ambos tienen procesos de aprendizaje supervisados y no supervisados, el deep learning es parte del machine learning avanzado pero la diferencia principal es **que el deep learning utiliza redes neuronales y el machine learning utiliza arboles de decision**

1.3 3.0 Libreria Tensorflow

Es la libreria mas famosa y utilizada para el Deep Learning.

Facilita los medios para trabajar, entrenar e inferenciar las redes neuronales

Originaria de Google, su version estable es de 2017.

Desarrollada para mejorar los resultados de busqueda y recomendaciones

Se instala a traves de la sentencia *pip install tensorflow*

Se importa del siguiente modo:

```
import tensorflow as tf
```

1.3.1 Componentes de la libreria: Tensor

El **Tensor** son vectores o matrices de las dimensiones que sean, para representar los datos y sus operaciones.

Cumple estas características:

- Esta compuesto de datos de entrada nuevos o ya operados con anterioridad
- Todas las operaciones ocurren dentro de un grafo
- Cada una de las operaciones se llama *op node* y todos se interrelacionan entre si

1.3.2 Componentes de la libreria: Graphs

Los **Graphs** son la representación del conjunto de todas las operaciones realizadas durante el entrenamiento y para la generación del modelo, siendo el resultado de conectar todos los tensores entre ellos.

Cumple estas características:

- Todas las operaciones se realizan conectando tensores
- Los tensores hacen sus funciones de nodos conectados por un *Edge* (un facilitador de operación entre nodos). > El tensor realiza las operaciones matemáticas > > El edge explica la relación entre input/output entre nodos.

```
[1]: pip install tensorflow
```

```
Collecting tensorflow
  Downloading tensorflow-2.20.0-cp313-cp313-win_amd64.whl.metadata (4.6 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl.metadata (5.3 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (24.2)
Requirement already satisfied: protobuf>=5.28.0 in c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (5.29.3)
Requirement already satisfied: requests<3,>=2.21.0 in c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (72.1.0)
Requirement already satisfied: six>=1.12.0 in c:\users\insau\anaconda3\lib\site-
```

```

packages (from tensorflow) (1.17.0)
Collecting termcolor>=1.1.0 (from tensorflow)
  Downloading termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in
c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in
c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (1.17.0)
Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
  Downloading grpcio-1.78.1-cp313-cp313-win_amd64.whl.metadata (3.9 kB)
Collecting tensorboard~=2.20.0 (from tensorflow)
  Downloading tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
Collecting keras>=3.10.0 (from tensorflow)
  Downloading keras-3.13.2-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.26.0 in
c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (2.1.3)
Requirement already satisfied: h5py>=3.11.0 in
c:\users\insau\anaconda3\lib\site-packages (from tensorflow) (3.12.1)
Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)
  Downloading ml_dtypes-0.5.4-cp313-cp313-win_amd64.whl.metadata (9.2 kB)
Requirement already satisfied: charset-normalizer<4,>=2 in
c:\users\insau\anaconda3\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in
c:\users\insau\anaconda3\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
c:\users\insau\anaconda3\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
c:\users\insau\anaconda3\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2025.4.26)
Requirement already satisfied: markdown>=2.6.8 in
c:\users\insau\anaconda3\lib\site-packages (from
tensorboard~=2.20.0->tensorflow) (3.8)
Requirement already satisfied: pillow in c:\users\insau\anaconda3\lib\site-
packages (from tensorboard~=2.20.0->tensorflow) (11.1.0)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from
tensorboard~=2.20.0->tensorflow)
  Downloading tensorboard_data_server-0.7.2-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: werkzeug>=1.0.1 in
c:\users\insau\anaconda3\lib\site-packages (from
tensorboard~=2.20.0->tensorflow) (3.1.3)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
c:\users\insau\anaconda3\lib\site-packages (from astunparse>=1.6.0->tensorflow)
(0.45.1)
Requirement already satisfied: rich in c:\users\insau\anaconda3\lib\site-
packages (from keras>=3.10.0->tensorflow) (13.9.4)
Collecting namex (from keras>=3.10.0->tensorflow)

```

```

Downloading namex-0.1.0-py3-none-any.whl.metadata (322 bytes)
Collecting optree (from keras>=3.10.0->tensorflow)
  Downloading optree-0.19.0-cp313-cp313-win_amd64.whl.metadata (35 kB)
Requirement already satisfied: MarkupSafe>=2.1.1 in
c:\users\insau\anaconda3\lib\site-packages (from
werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in
c:\users\insau\anaconda3\lib\site-packages (from
rich->keras>=3.10.0->tensorflow) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
c:\users\insau\anaconda3\lib\site-packages (from
rich->keras>=3.10.0->tensorflow) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in c:\users\insau\anaconda3\lib\site-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.0)
Downloading tensorflow-2.20.0-cp313-cp313-win_amd64.whl (332.0 MB)
----- 0.0/332.0 MB ? eta -:-:--
- ----- 12.1/332.0 MB 63.1 MB/s eta 0:00:06
-- ----- 24.1/332.0 MB 59.0 MB/s eta 0:00:06
----- 35.1/332.0 MB 57.5 MB/s eta 0:00:06
----- 50.1/332.0 MB 60.2 MB/s eta 0:00:05
----- 65.0/332.0 MB 62.5 MB/s eta 0:00:05
----- 80.5/332.0 MB 63.4 MB/s eta 0:00:04
----- 93.8/332.0 MB 64.3 MB/s eta 0:00:04
----- 108.8/332.0 MB 64.4 MB/s eta 0:00:04
----- 123.5/332.0 MB 64.8 MB/s eta 0:00:04
----- 138.7/332.0 MB 65.3 MB/s eta 0:00:03
----- 154.7/332.0 MB 65.8 MB/s eta 0:00:03
----- 170.7/332.0 MB 66.5 MB/s eta 0:00:03
----- 185.3/332.0 MB 66.6 MB/s eta 0:00:03
----- 202.1/332.0 MB 67.4 MB/s eta 0:00:02
----- 218.6/332.0 MB 68.0 MB/s eta 0:00:02
----- 227.8/332.0 MB 66.5 MB/s eta 0:00:02
----- 238.6/332.0 MB 65.5 MB/s eta 0:00:02
----- 249.3/332.0 MB 64.8 MB/s eta 0:00:02
----- 263.2/332.0 MB 64.9 MB/s eta 0:00:02
----- 277.1/332.0 MB 64.8 MB/s eta 0:00:01
----- 288.1/332.0 MB 64.9 MB/s eta 0:00:01
----- 298.6/332.0 MB 64.1 MB/s eta 0:00:01
----- 309.3/332.0 MB 63.3 MB/s eta 0:00:01
----- 319.3/332.0 MB 62.3 MB/s eta 0:00:01
----- 330.3/332.0 MB 61.6 MB/s eta 0:00:01
----- 331.9/332.0 MB 61.5 MB/s eta 0:00:01
----- 331.9/332.0 MB 61.5 MB/s eta 0:00:01
----- 332.0/332.0 MB 54.7 MB/s eta 0:00:00
Downloading grpcio-1.78.1-cp313-cp313-win_amd64.whl (4.8 MB)
----- 0.0/4.8 MB ? eta -:-:--
----- 4.8/4.8 MB 54.3 MB/s eta 0:00:00
Downloading ml_dtypes-0.5.4-cp313-cp313-win_amd64.whl (212 kB)

```

```

Downloading tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
----- 0.0/5.5 MB ? eta -:--:--
----- 5.5/5.5 MB 47.6 MB/s eta 0:00:00
Downloading tensorboard_data_server-0.7.2-py3-none-any.whl (2.4 kB)
Downloading absl_py-2.4.0-py3-none-any.whl (135 kB)
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading gast-0.7.0-py3-none-any.whl (22 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading keras-3.13.2-py3-none-any.whl (1.5 MB)
----- 0.0/1.5 MB ? eta -:--:--
----- 1.5/1.5 MB 50.5 MB/s eta 0:00:00
Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl (26.4 MB)
----- 0.0/26.4 MB ? eta -:--:--
----- 12.6/26.4 MB 58.7 MB/s eta 0:00:01
----- 25.7/26.4 MB 60.6 MB/s eta 0:00:01
----- 26.4/26.4 MB 52.8 MB/s eta 0:00:00
Downloading opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Downloading namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading optree-0.19.0-cp313-cp313-win_amd64.whl (337 kB)
Installing collected packages: namex, libclang, flatbuffers, termcolor,
tensorboard-data-server, optree, opt_einsum, ml_dtypes, grpcio, google_pasta,
gast, astunparse, absl-py, tensorboard, keras, tensorflow

```

```

-- ----- 1/16 [libclang]
-- ----- 1/16 [libclang]
-- ----- 1/16 [libclang]
-- ----- 1/16 [libclang]
-- ----- 1/16 [libclang]
----- ----- 2/16 [flatbuffers]
----- ----- 5/16 [optree]
----- ----- 6/16 [opt_einsum]
----- ----- 6/16 [opt_einsum]
----- ----- 8/16 [grpcio]
----- ----- 8/16 [grpcio]
----- ----- 8/16 [grpcio]
----- ----- 8/16 [grpcio]
----- ----- 9/16 [google_pasta]
----- ----- 10/16 [gast]
----- ----- 12/16 [absl-py]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]
----- ----- 13/16 [tensorboard]

```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Successfully installed absl-py-2.4.0 astunparse-1.6.3 flatbuffers-25.12.19
gast-0.7.0 google_pasta-0.2.0 grpcio-1.78.1 keras-3.13.2 libclang-18.1.1
ml_dtypes-0.5.4 namex-0.1.0 opt_einsum-3.4.0 optree-0.19.0 tensorboard-2.20.0
tensorboard-data-server-0.7.2 tensorflow-2.20.0 termcolor-3.3.0
Note: you may need to restart the kernel to use updated packages.

1.3.3 3.1 El motor de Tensorflow: Keras

La libreria TensorFlow, lleva incluida en su programacion una API dedicada a la ejecucion de los procesos que imitan las redes neuronales, llamada *Keras*

Esta herramienta **hace sus veces de interfaz para la libreria Tensorflow**

Esta herramienta incluye multiples herramientas:

- Para el preprocesado de datos
- Para la generacion de modelos
- Para la gestion de datos de entrada y salida
- Para la gestion de las redes neuronales y sus capas

Hay que importarlo a nuestro entorno para hacer uso de el

from tensorflow import keras

1.4 Ejemplo para probar TensorFlow

```
[7]: # 0. Hay que traer al entorno la libreria Tensorflow
      # recomendable traer Matplotlib y numpy
```

```
from tensorflow import keras
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

```
[3]: # 1. Traemos un dataset preparado para generar un modelo de reconocimiento de
      ↪ropa. (son 70.000 imagenes en matrices)
      # Cargamos el dataset
      # Preparamos 2 dataset (90%/10%) uno de entrenamiento y otro de testeo
      # Los datos llevan etiquetas que identifican que prendas son

fashion_mnist = tf.keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.
↪load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-labels-idx1-ubyte.gz
29515/29515          0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-images-idx3-ubyte.gz
26421880/26421880    1s
0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-labels-idx1-ubyte.gz
5148/5148           0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-images-idx3-ubyte.gz
4422102/4422102      0s
0us/step
```

```
[8]: # 2. Generamos clases para cada etiqueta que existe en nuestro dataset
```

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
[13]: #3. Miramos los datos de entrenamieno
```

```
train_images.shape
```

```
[13]: (60000, 28, 28)
```

```
[14]: #3. Miramos los datos de test
```

```
test_images.shape
```

```
[14]: (10000, 28, 28)
```

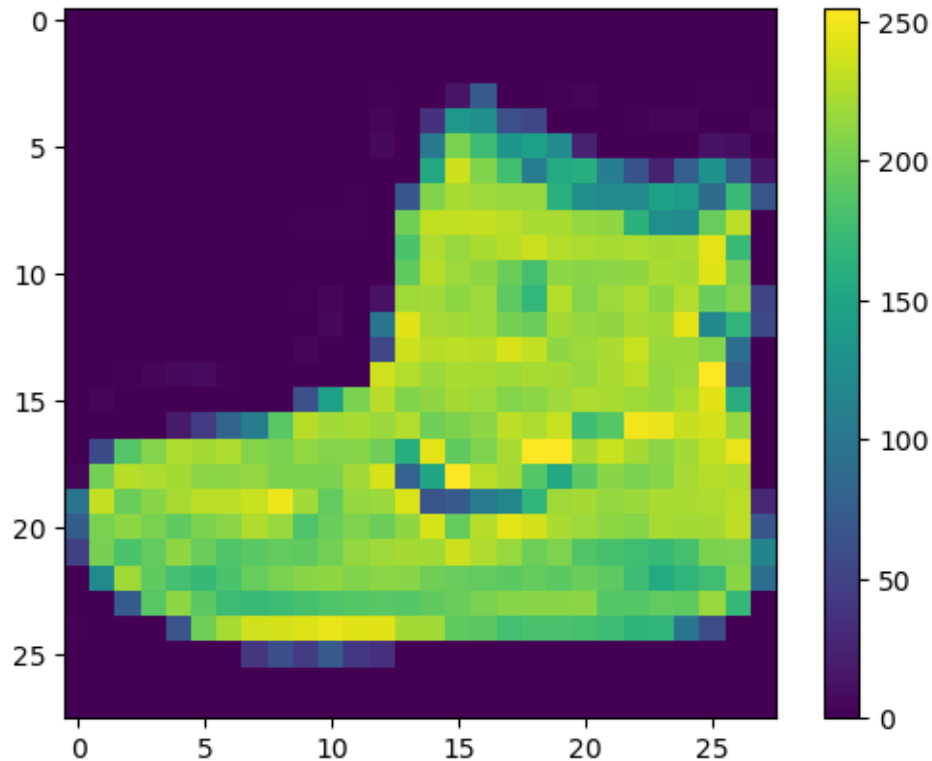
```
[15]: # 4.0 Preprocesamos los datos: Explicacion y ejemplo
```

```
    # Es el proceso de convertir la informacion a un lenguaje interpretable_  
    ↪ por nuestro sistema
```

```
    # En este caso al ser imagenes, va a pasar a mapas de bits compuestos por_  
    ↪ informacion binaria
```

```
    # Aqui vemos un ejemplo de ejecucion de que les ocurre a las imagenes_  
    ↪ cuando operamos con ellas (en este caso generamos un grafico)
```

```
plt.figure()  
plt.imshow(train_images[0])  
plt.colorbar()  
plt.grid(False)  
plt.show()
```

```
[16]: # 4.1 Preprocesado de datos: Normalizacion de datos
      # Es el proceso de transformacion de la escala de los datos
      # Recuerda que los valores deben estar comprendidos entre 0 y 1 para ser
      ↪ interpretados
      # Se realiza este proceso, dividiendo el conjunto de valores por el n°
      ↪ total de bits (255 es el maximo de bits por pixel)
      # Tanto el conjunto de entrenamiento como el test se deben convertir a la
      ↪ misma escala

train_images = train_images / 255.0
test_images = test_images / 255.0
```

```
[17]: # 4.2 Preprocesado de datos: Comprobamos datos normalizados
      # En este caso transformamos las imagenes de binario a mapa de bits
      # Incluimos la etiqueta en base a una de las clases que llevan su mismo
      ↪ nombre

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
```

```
plt.yticks([])
plt.grid(False)
plt.imshow(train_images[i], cmap=plt.cm.binary)
plt.xlabel(class_names[train_labels[i]])
plt.show()
```



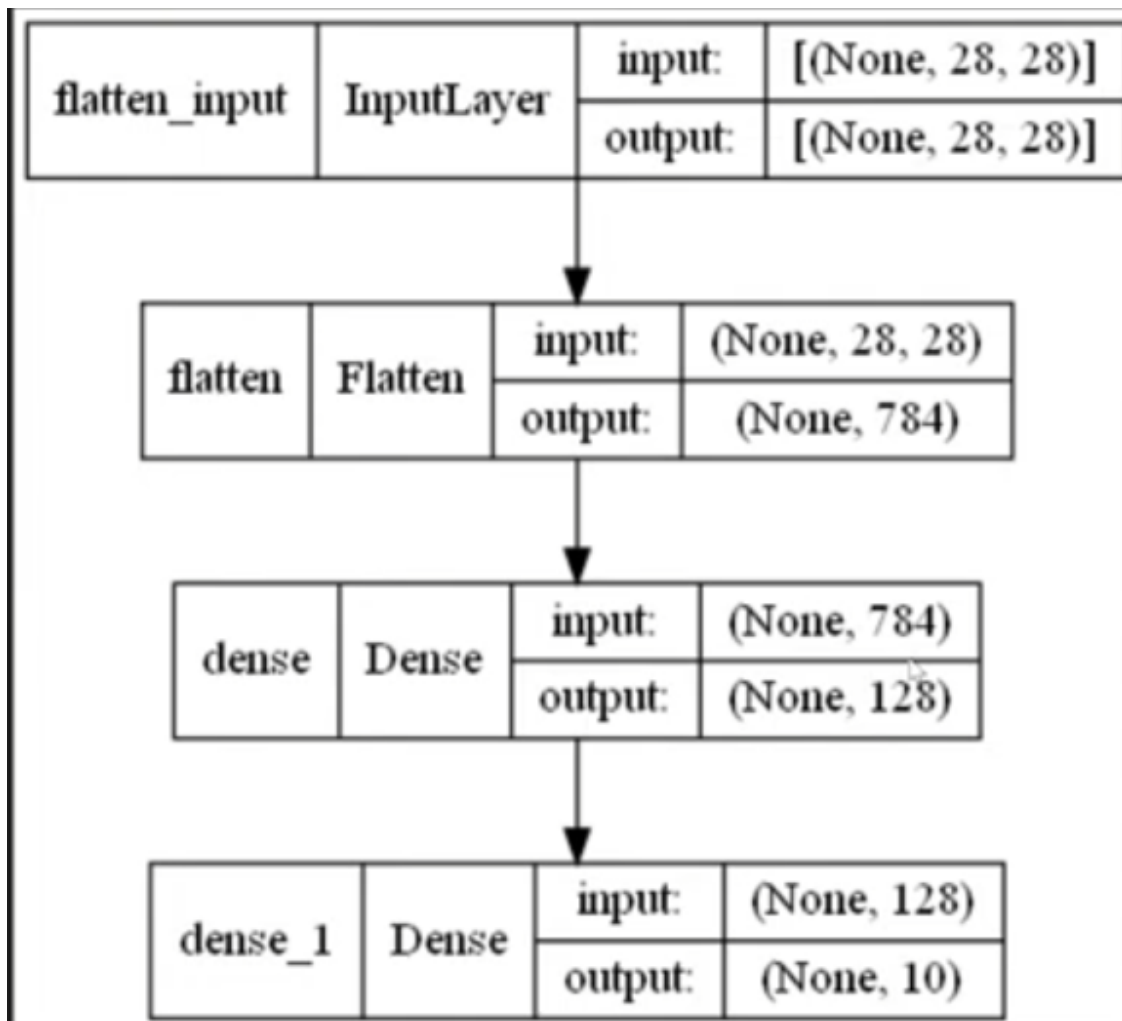
[21]: # 5. Definicion del modelo a traves de keras.
 # Lo realizaremos a traves de una funcion de keras que sea secuencial: tf.
 ↪keras.Sequential ()
 # Esta funcion incluye un parametro que cambia la forma del Array y lo
 ↪aplana. [era 28 x 28 y se convierte en un vector (de 784 valores en este
 ↪caso)]

Incluye otro parametro para definir la densidad de las capas. En este caso se define se han definido 2 capas:

una 1ª capa de nodos de filtrado amplia (128 nodos)

una 2ª capa para la clasificacion en base a las etiquetas (10 etiquetas = 10 nodos)

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
```



[22]: # 6.0 Compilamos el modelo
Es una funcion que permite configurar mas el modelo antes de entrenarlo: .
compile()
Dispone de 3 parametros:

```

    # optimizer = así es como se actualiza el modelo en función de los
    ↪ datos que ve y su función de pérdida.
    # loss = mide la precisión del modelo durante el entrenamiento.
    # metrics = se utilizan para supervisar los pasos de entrenamiento y
    ↪ prueba.

model.compile(optimizer='adam',
              loss=tf.keras.losses.
    ↪ SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

```

```

[23]: # 7.0 Entrenamos el modelo
      # Utilizaremos el dataset de entrenamiento preparado anteriormente + la
      ↪ funcion .fit()
      # Hay que indicarle cuantas veces queremos que repita el aprendizaje para
      ↪ mejorar el ajuste: epochs (10 en este caso)
      # Generara un ajuste entre 0 y 1 lo mas optimo posible, en base al
      ↪ compilador anterior, sobre los datos del dataset de entrenamiento

model.fit(train_images, train_labels, epochs=10)

```

```

Epoch 1/10
1875/1875          11s 5ms/step -
accuracy: 0.8252 - loss: 0.4998
Epoch 2/10
1875/1875          10s 5ms/step -
accuracy: 0.8647 - loss: 0.3775
Epoch 3/10
1875/1875          10s 5ms/step -
accuracy: 0.8761 - loss: 0.3382
Epoch 4/10
1875/1875          10s 5ms/step -
accuracy: 0.8849 - loss: 0.3140
Epoch 5/10
1875/1875          10s 5ms/step -
accuracy: 0.8913 - loss: 0.2955
Epoch 6/10
1875/1875          11s 5ms/step -
accuracy: 0.8956 - loss: 0.2814
Epoch 7/10
1875/1875          11s 5ms/step -
accuracy: 0.9002 - loss: 0.2678
Epoch 8/10
1875/1875          10s 5ms/step -
accuracy: 0.9048 - loss: 0.2564
Epoch 9/10
1875/1875          11s 5ms/step -

```

```
accuracy: 0.9075 - loss: 0.2461
Epoch 10/10
1875/1875          9s 5ms/step -
accuracy: 0.9092 - loss: 0.2393
```

[23]: <keras.src.callbacks.history.History at 0x197d9148980>

```
[24]: # 8.0 Evaluar la precision del modelo
      # Una vez que ya lo hemos entrenado con el dataset de entrenamiento el
      ↪ siguiente paso es testear con el dataset de test.
      # Para ello haremos uso de la funcion .evaluate()

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

print('\nTest accuracy:', test_acc)
```

```
313/313 - 1s - 3ms/step - accuracy: 0.8791 - loss: 0.3520
```

Test accuracy: 0.8791000247001648

```
[27]: # 9.0 Realizar predicciones con el modelo
      # Una vez realizado el modelo secuencial y testeado podemos utilizar otro
      ↪ dataset para predecir que prendas son
      # Incluimos una funcion tf.keras.layers.Softmax() que traduzca los datos
      ↪ obtenidos (logits) para su correcta interpretacion
      # Utilizaremos la funcion .predict() para generar predicciones de nuestro
      ↪ modelo

probability_model = tf.keras.Sequential([model,
                                         tf.keras.layers.Softmax()])

predictions = probability_model.predict(test_images)

predictions[0]
```

```
313/313          1s 3ms/step
```

```
[27]: array([4.0579604e-08, 1.7654513e-13, 1.4969793e-09, 2.4148681e-09,
            2.5691040e-09, 3.8889175e-05, 5.6035695e-08, 3.6144731e-04,
            3.6530609e-10, 9.9959964e-01], dtype=float32)
```

```
[28]: # 10.1 Verificamos las predicciones: Generamos una funcion que representa y
      ↪ evalua la precision de las etiquetas predichas
      # Revisara que etiqueta predicha es el mas proximo a la etiqueta asociada
      ↪ (hay 10 etiquetas, del 0 al 9)
      # Indicara en color azul las etiquetas de prediccion correctas y en rojo
      ↪ las incorrectas
```

```

def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'

    plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                        100*np.max(predictions_array),
                                        class_names[true_label]),
              color=color)

def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')

```

[29]: *# 10.2 Verificamos las predicciones: Usamos la funcion definida para que evalúe distintas imagenes predichas*

Plot the first X test images, their predicted labels, and the true labels.
Color correct predictions in blue and incorrect predictions in red.
Indica el % de precision sobre la prediccion.

```

num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, predictions[i], test_labels, test_images)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, predictions[i], test_labels)
plt.tight_layout()

```

```
plt.show()
```

